←

چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۶

کاتلین پیشرفته — generics، توابع inline و reified، DSLها، context parameterها، K2 و KSP



اسماعیل مرادی

@esmaeeil-moradi700

چرا کاتلین پیشرفته روز بیست‌وششم است؟

مصاحبه‌های سینیور نمی‌پرسند «data class چیست؟». می‌پرسند **چرا** زبان این‌طور رفتار می‌کند — در زمان کامپایل چه اتفاقی می‌افتد، چه چیزی تا زمان اجرا باقی می‌ماند و هزینه‌اش چیست:

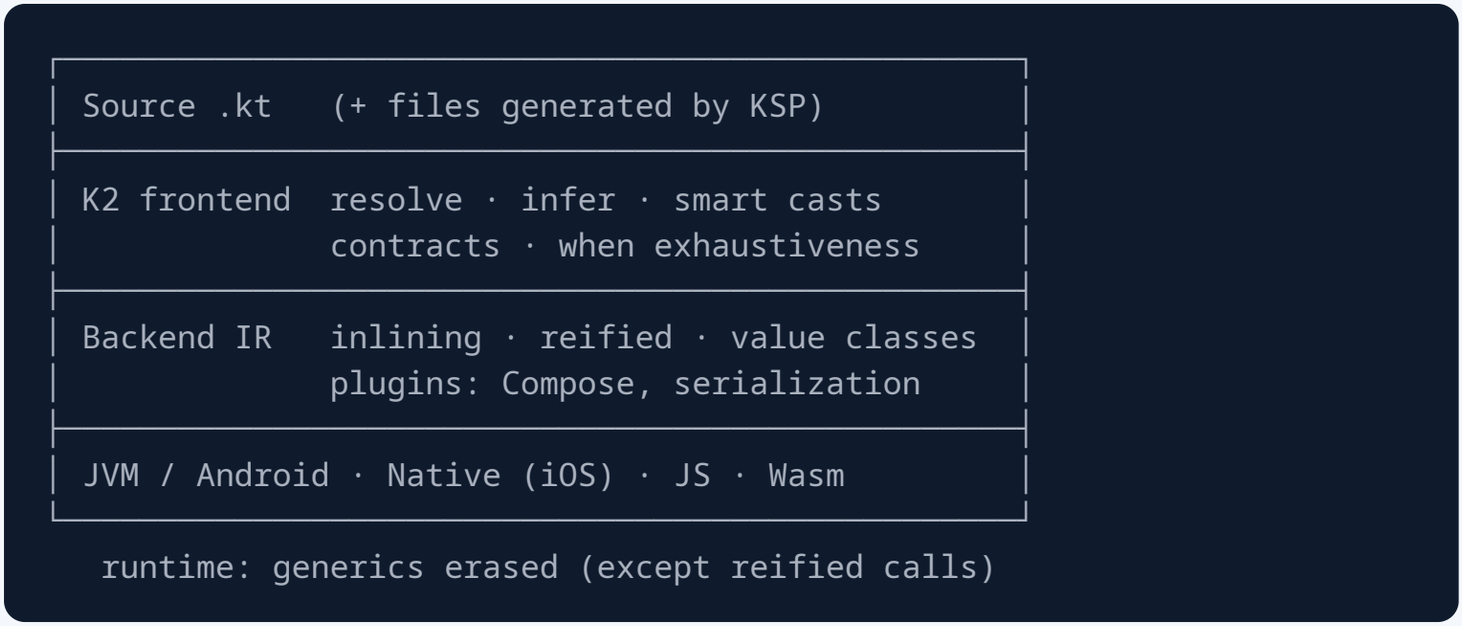
- `inline` و `reified` و type erasure — فیلتر کلاسیک «bytecode را توضیح بده»
- sealed class و value class برای مدل‌های domain با type safety
- variance: چرا `List<String>` یک `List<Any>` هست ولی `MutableList<String>` نیست
- از Kotlin 2.0 کامپایلر پیش‌فرض K2 است؛ kapt در حالت نگهداری است و KSP استاندارد شده
- KMP موضوع `expect` و `actual` و طراحی API عمومی را وسط می‌آورد

سطح	سؤال‌ها
جونیور	توابع inline، reified و erasure، سلسله‌مراتب sealed
مید	generics و variance، value classها، delegation، DSLها
سینیور	context parameterها، contractها و smart cast، کامپایلر K2
آرکیتکت	KSP و متاپروگرمینگ، expect و actual، طراحی API عمومی



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: زمان کامپایل در برابر زمان اجرا



- بیشتر قابلیت‌های «پیشرفته» مربوط به **زمان کامپایل** هستند: در زمان اجرا هزینه‌ای ندارند — تا وقتی چیزی boxing یا allocation را اجباری کند
- روی JVM، آرگومان‌های تایپ generic **پاک (erase)** می‌شوند؛ فقط ترکیب inline و `reified` آن‌ها را، برای هر محل فراخوانی، برمی‌گرداند
- کامپایلر به بعضی اعلان‌ها **اعتماد** می‌کند (contractها، `@UnsafeVariance`، و `@Stable` در Compose) — قول غلط یعنی باگ خودتان

سؤالی که پشت همهٔ جواب‌های امروز است

«این در زمان اجرا وجود دارد، یا فقط در کامپایلر؟» — inline، reified، value classها، contractها و KSP همه با جواب همین سؤال معنا پیدا می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

توابع inline، reified و erasure، سلسله‌مراتب sealed



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. توابع inline: واقعاً چه چیزی می‌شود

```kotlin
inline fun <T> measure(tag: String, block: () -> T): T {
    val start = System.nanoTime()
    try {
        return block()
    } finally {
        Log.d(tag, "took ${System.nanoTime() - start} ns")
    }
}

fun findUser(users: List<User>, id: String): User? {
    users.forEach { user ->
        if (user.id == id) return user // non-local return
    }
    return null
}
```

- کامپایلر بدنهٔ تابع **و lambda** را در محل فراخوانی کپی می‌کند: نه function objectی ساخته می‌شود، نه فراخوانی اضافه‌ای
- lambdaهایی که به تابع inline داده می‌شوند می‌توانند **`return` غیرمحلی** داشته باشند — `forEach` تابعی inline است، پس `return` `user` از خود `findUser` خارج می‌شود
- همین inline شدن است که `reified` را ممکن می‌کند (سؤال ۲)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. کلیدواژه‌های noinline و crossinline

پارامتر	معنا	کِی
حالت پیش‌فرض	lambda اینلاین می‌شود	بدون allocation؛ `return` غیرمحلی مجاز است
`noinline`	lambda یک **object واقعی** باقی می‌ماند	وقتی آن را نگه می‌دارید یا به تابعی غیر inline پاس می‌دهید
`crossinline`	اینلاین می‌شود، ولی `return` غیرمحلی ممنوع است	وقتی lambda داخل یک lambda یا object دیگر اجرا می‌شود

```
inline fun View.onClick(crossinline action: () -> Unit) {
    setOnClickListener { action() } // runs in another lambda
}
```

تلهٔ مصاحبه

«`noinline` *از ساخته شدن object برای lambda جلوگیری می‌کند.*» — برعکس است، و نسخهٔ پارسال همین را گفته بود. `noinline` آن object را **می‌سازد**، چون به مقداری نیاز دارید که بشود نگهش داشت یا جای دیگری پاسش داد.

تلهٔ مصاحبه

«*برای سرعت همه چیز را inline کنید.*» — بدون پارامتر lambda سودش ناچیز است و IDE هم هشدار می‌دهد؛ هر محل فراخوانی هم bytecode را بزرگ‌تر می‌کند. بدنهٔ یک تابع inline عمومی هم در کد مصرف‌کننده کپی می‌شود — تغییرش یعنی آن‌ها باید دوباره کامپایل کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. type erasure و reified

```
// Doesn't compile: "Cannot check for instance of erased type"
fun <T> List<Any>.firstOfType(): T? =
    firstOrNull { it is T } as T?

inline fun <reified T> List<Any>.firstOfType(): T? =
    firstOrNull { it is T } as T?

inline fun <reified T : Activity> Context.open(
    vararg extras: Pair<String, Any?>,
) {
    startActivity(
        Intent(this, T::class.java).putExtras(bundleOf(*extras))
    )
}

context.open<DetailActivity>("id" to 42L)
```

- روی JVM، `List<String>` و `List<Int>` در زمان اجرا **یک کلاس** هستند
- `reified` فقط در توابع **inline** کار می‌کند: کامپایلر در هر محل فراخوانی تایپ واقعی را جایگزین می‌کند، پس `is T` و `T::class` و `typeOf<T>()` کار می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«`reified` *تایپ‌های generic را در زمان اجرا نگه می‌دارد.*» — فقط داخل همان فراخوانی inline. کلاس‌ها همچنان erase می‌شوند؛ تابع reified را نمی‌شود از Java صدا زد، و `T` باید در محل فراخوانی معلوم باشد.

تلهٔ مصاحبه

«`T::class` *برای* `List<User>` *کافی است.*» — فقط `List` را می‌دهد، نه `List<User>`. برای آرگومان‌های تایپ تودرتو از `typeOf<T>()` استفاده کنید — `Json.decodeFromString<List<User>>(json)` هم دقیقاً همین‌طور serializer درست را پیدا می‌کند.

سؤال تکمیلی

«*هر روز کجا از reified استفاده می‌کنید؟*» — در `filterIsInstance<T>()`، `viewModel<T>()`، تابع `get<T>()` در Koin، `decodeFromString<T>()` در kotlinx.serialization و `body<T>()` در Ktor.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. سلسله‌مراتب sealed و when کامل

```
sealed interface PaymentResult {
    data class Success(val receiptId: String) : PaymentResult
    data class Declined(val reason: String) : PaymentResult
    data object Cancelled : PaymentResult
}

fun message(r: PaymentResult): String = when (r) {
    is PaymentResult.Success -> "Paid: ${r.receiptId}"
    is PaymentResult.Declined if r.reason == "3DS" ->
        "Please confirm in your bank app"
    is PaymentResult.Declined -> "Declined: ${r.reason}"
    PaymentResult.Cancelled -> "Cancelled"
}
```

	enum	sealed class	sealed interface
نمونه‌ها	singletonهای ثابت	هر تعداد، با state خودشان	هر تعداد، با state خودشان
زیرتایپ‌ها چه دارند	propertyهای یکسان	دادهٔ متفاوت	دادهٔ متفاوت
constructor یا state مشترک	دارد	دارد	ندارد
یک کلاس چند تا از آن داشته باشد	—	نه (فقط یک superclass)	بله



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. تله‌های sealed class

تلهٔ مصاحبه

«*اگر `when` کامل نباشد، کامپایلر هشدار می‌دهد.*» — نسخهٔ پارسال همین را گفته بود. از Kotlin 1.7، `when` ناقص روی تایپ sealed، enum یا `Boolean` یک **خطای** کامپایل است — هم برای statement و هم برای expression.

تلهٔ مصاحبه

اضافه کردن `else ->` به `when` روی تایپ sealed. بررسی کامل بودن را خاموش می‌کند: زیرتایپ بعدی که کسی اضافه کند بی‌صدا به `else` می‌افتد، به‌جای اینکه build را fail کند.

- زیرکلاس‌های یک تایپ sealed باید در **همان ماژول و همان package** باشند
- **guard condition**ها (مثل `is X if cond ->` در اسلاید قبل) از Kotlin 2.2 پایدار (stable) هستند

سؤال تکمیلی

«*sealed class یا sealed interface؟*» — پیش‌فرض interface: انعطاف بیشتر، و یک تایپ می‌تواند چند تا را پیاده کند. sealed class وقتی زیرتایپ‌ها state مشترک در constructor دارند.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

generics و variance، value classها، delegation، DSLها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. variance: کلیدواژه‌های in و out، و حالت invariant

```kotlin
interface Source<out T> {        // producer: only returns T
    fun next(): T
}

interface Sink<in T> {           // consumer: only accepts T
    fun put(item: T)
}

fun demo(strings: Source<String>, anySink: Sink<Any>) {
    val a: Source<Any> = strings    // OK: out = covariant
    val b: Sink<String> = anySink   // OK: in = contravariant
}
```

کاتلین	جاوا	معنا
`out T`	`? extends T`	تولیدکننده — می‌توانید `T` را بخوانید
`in T`	`? super T`	مصرف‌کننده — می‌توانید `T` را پاس بدهید
`*`	`?`	تایپ نامعلوم — به شکل `Any?` خوانده می‌شود، نوشتن ممنوع
`T`	`T`	invariant — هم خواندن و هم نوشتن



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. variance در محل استفاده، و boundها

```
// Use-site variance: this call only reads `from`
fun copy(from: Array<out Number>, to: Array<in Number>) {
    for (i in from.indices) to[i] = from[i]
}

// Several upper bounds
fun <T> sortAll(items: MutableList<T>)
    where T : Named, T : Comparable<T> = items.sort()

// Definitely non-nullable: T & Any
fun <T> T.orThrow(): T & Any = this ?: error("null")
```

تلهٔ مصاحبه

«`MutableList<String>` یک `MutableList<Any>` *است.*» — **نه:** invariant است، وگرنه می‌توانستید یک `Int` به لیست رشته‌ها اضافه کنید. اما `List<String>` **یک** `List<Any>` هست، چون به شکل `List<out E>` تعریف شده و فقط‌خواندنی است.

سؤال تکمیلی

«پس `List<out E>` *چطور متد* `contains(element: E)` *دارد؟*» — کتابخانهٔ استاندارد آن را با `@UnsafeVariance` علامت زده: `contains` فقط می‌خواند، پس این موقعیت ناامن عمداً مجاز شده است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. value classها: type safety بدون object پوشاننده

```
@JvmInline
value class UserId(val raw: String) {
    init { require(raw.isNotBlank()) { "blank id" } }
}

@JvmInline
value class Cents(val value: Long) {
    operator fun plus(other: Cents) = Cents(value + other.value)
}

fun load(id: UserId) { /* an OrderId can't be passed here */ }
```

- هر جا با تایپ استاتیک خودش استفاده شود، به مقدار زیرین (`String` یا `long`) کامپایل می‌شود
- وقتی **box** می‌شود که به‌عنوان آرگومان تایپ generic مثل `List<UserId>`، نسخهٔ nullable یک تایپ اولیه مثل `Cents?`، یا به شکل `Any` یا یک interface استفاده شود
- اسم JVM توابعی که value class می‌گیرند **mangle** می‌شود — صدا زدنشان از Java دردسر دارد

تلهٔ مصاحبه

«*inline classها هیچ سربازی ندارند.*» — نسخهٔ پارسال همین را گفته بود. فقط وقتی box نشوند (بالا را ببینید). و `inline class` سینتکس قدیمی و deprecated است: از `@JvmInline value class` استفاده کنید. مقایسهٔ هویت (`===`) مجاز نیست؛ برابری بر اساس مقدار است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. property delegation

```
class SavedDelegate<T>(
    private val handle: SavedStateHandle,
    private val default: T,
) : ReadWriteProperty<Any?, T> {
    override fun getValue(
        thisRef: Any?, property: KProperty<*>,
    ): T = handle.get<T>(property.name) ?: default

    override fun setValue(
        thisRef: Any?, property: KProperty<*>, value: T,
    ) {
        handle[property.name] = value
    }
}

class SearchViewModel(handle: SavedStateHandle) : ViewModel() {
    var query: String by SavedDelegate(handle, "")
}
```

- کلیدواژهٔ `by` getter و setter یک property را به object دیگری می‌سپارد: همان عملگرهای `getValue` و `setValue`
- نمونه‌های آماده: `lazy`، `Delegates.observable` و `vetoable`، `by map`، و سپردن به property دیگر (`by ::other`)
- اندروید همه جا از آن استفاده می‌کند: `by viewModels()` و `by remember { mutableStateOf(…) }`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. class delegation — و تله‌هایش

```
class LoggingList<T>(
    private val inner: MutableList<T> = mutableListOf(),
) : MutableList<T> by inner {
    override fun add(element: T): Boolean {
        println("add $element")
        return inner.add(element)
    }
}
```

تلهٔ مصاحبه

«`addAll()` از `add()` *بازنویسی‌شدهٔ من عبور می‌کند.*» — **نه.** delegation ارث‌بری نیست: `addAll` به `inner` فرستاده می‌شود و **همان‌جا** انجام می‌شود. override شما هیچ‌وقت آن عنصرها را نمی‌بیند.

تلهٔ مصاحبه

`lazy { }` به‌صورت پیش‌فرض `SYNCHRONIZED` است — thread-safe، با یک قفل. `LazyThreadSafetyMode.PUBLICATION` یا `NONE` را فقط وقتی به کار ببرید که مدل threading را می‌دانید، نه به‌صورت پیش‌فرض «برای سرعت».



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. builderهای type-safe یا همان DSLها

```
@DslMarker annotation class FormDsl

@FormDsl
class FormBuilder {
    private val fields = mutableListOf<Field>()
    fun text(name: String, init: FieldBuilder.() -> Unit = {}) {
        fields += FieldBuilder(name).apply(init).build()
    }
    fun build() = Form(fields.toList())
}

@FormDsl
class FieldBuilder(private val name: String) {
    var required = false
    var maxLength = Int.MAX_VALUE
    fun build() = Field(name, required, maxLength)
}

fun form(block: FormBuilder.() -> Unit): Form =
    FormBuilder().apply(block).build()
```

- هستهٔ آن **lambda with receiver** است، یعنی `FormBuilder.() ->` داخل آن، `this` همان builder است: `Unit`
- extension function، `infix` و `operator invoke` را وقتی اضافه کنید که محل فراخوانی را طبیعی‌تر می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. استفاده از DSL — و کاری که `@DslMarker` واقعاً می‌کند

```
val signUp = form {
    text("email") { required = true }
    text("bio") {
        maxLength = 160
        text("nested")          // error: implicit outer receiver
        this@form.text("x")     // compiles: explicit receiver
    }
}
```

تلهٔ مصاحبه

«`@DslMarker` *تودرتوی غلط را غیرممکن می‌کند.*» — فقط دسترسی **ضمنی** به receiverهای بیرونی را می‌بندد. یک `this@form` صریح همچنان کامپایل می‌شود — نسخهٔ پارسال در این مورد اغراق کرده بود.

تلهٔ مصاحبه

DSLی که در حین ساخته شدن کار انجام می‌دهد (فراخوانی شبکه، نوشتن در دیتابیس). اول یک **مدل تغییرناپذیر** بسازید، بعد آن را تفسیر یا اعتبارسنجی کنید — هم تست‌پذیر است و هم فهمیدنش ساده‌تر.

سؤال تکمیلی

«*آیا Compose یک builder DSL است؟*» — نه. composableها تابع‌هایی هستند که پلاگین کامپایلر Compose آن‌ها را تبدیل می‌کند. البته scopeهایش (`RowScope` و `ColumnScope`) از یک `@DslMarker` استفاده می‌کنند — یعنی `@LayoutScopeMarker` .



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

context parameterها، contractها و smart cast، کامپایلر K2



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. context parameterها

KOTLIN 2.2 (PREVIEW)

context parameterها در Kotlin 2.2 به شکل **preview** آمدند و با فلگ کامپایلر `-Xcontext-parameters` فعال می‌شوند. جایگزین **context receiver**های آزمایشی هستند که deprecated شده‌اند. قبل از استفاده در یک API عمومی، release note نسخهٔ Kotlin خودتان را چک کنید.

```kotlin
interface Logger { fun info(msg: String) }
interface Clock { fun now(): Long }

context(logger: Logger, clock: Clock)
fun startSync(job: SyncJob) {
    logger.info("Sync ${job.id} at ${clock.now()}")
}

fun runSync(job: SyncJob, logger: Logger, clock: Clock) {
    with(logger) {
        with(clock) {
            startSync(job) // both resolved from context
        }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. receiver یا parameter — و کِی از آن‌ها استفاده کنیم

	context receiverهای (deprecated)	context parameterها
شکل تعریف	`context(Logger)`	`context(logger: Logger)`
اعضا داخل تابع	به‌صورت ضمنی در دسترس‌اند: `info(…)`	با اسم صدا زده می‌شوند: `logger.info(…)`؛ و `_` اگر بی‌نام باشد
وضعیت	آزمایشی، deprecated	در حالت preview از Kotlin 2.2

- کاربرد درست: **قابلیت‌های عرضی (cross-cutting)** که از میان فراخوانی‌های زیادی عبور می‌کنند — logging، یک تراکنش، یک scope برای خطای تایپ‌دار
- جایگزین constructor injection برای سرویس‌ها نیست (روز ۷)

تلهٔ مصاحبه

«*context parameterها همان context receiverها با اسم جدیدند.*» — یک طراحی دوباره هستند: context parameter یک `this` ضمنی **نیست**، پس اعضایش به بدنهٔ تابع نشت نمی‌کنند.

تلهٔ مصاحبه

عبور دادن ده‌ها قابلیت با context parameter «تا DI لازم نباشد». حالا هر صدازننده‌ای باید همه را فراهم کند — وابستگی پنهانی که دنبال کردنش از یک constructor سخت‌تر است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. contractها: یاد دادن به کامپایلر

```
@OptIn(ExperimentalContracts::class)
fun Order?.isPayable(): Boolean {
    contract { returns(true) implies (this@isPayable != null) }
    return this != null && status == Status.OPEN
}

fun pay(order: Order?) {
    if (order.isPayable()) {
        submit(order.id) // smart cast: Order, not Order?
    }
}
```

- حالت `returns(…) implies (…)`: بعد از فراخوانی smart cast می‌گیرید — مثل `isNullOrEmpty()` و `require()` در کتابخانهٔ استاندارد
- حالت `callsInPlace(block, EXACTLY_ONCE)`: lambda همین‌جا و دقیقاً یک بار اجرا می‌شود — پس می‌شود یک `val` را داخلش مقداردهی کرد، مثل `run { }`
- هنوز پشت opt-in یعنی `@OptIn(ExperimentalContracts::class)` است

تلهٔ مصاحبه

«*کامپایلر contract من را بررسی می‌کند.*» — **نمی‌کند.** به آن اعتماد می‌کند. contract غلط روی مقداری که می‌تواند null باشد smart cast به non-null می‌دهد — crashی که سیستم تایپ قول داده بود رخ نمی‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. کامپایلر K2: چه چیزی عوض شد

- **Kotlin 2.0** کامپایلر K2 را پایدار و پیش‌فرض کرد: یک frontend جدید (FIR) و یک pipeline واحد برای JVM، Native، JS و Wasm
- **کامپایل سریع‌تر** در benchmarkهای build — JetBrains خودتان را قبل و بعد اندازه بگیرید
- **کامپایلر Compose** به ریپوی Kotlin منتقل شد: پلاگین `org.jetbrains.kotlin.plugin.compose`، هم‌نسخه با Kotlin
- **smart castهای هوشمندتر:**

```
// interface Ok : Status, interface Postponed : Status
fun handle(s: Any) {
    if (s is Ok || s is Postponed) {
        s.signal() // K2: smart cast to the common Status
    }
}

fun pet(animal: Any) {
    val isCat = animal is Cat
    if (isCat) animal.purr() // K2: smart cast via local val
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. مهاجرت یک کدبیس بزرگ به K2

حوزه	چه باید کرد
annotation processorها	مهاجرت از kapt به KSP (کتابخانه‌های Room، Moshi، Hilt و Dagger پشتیبانی می‌کنند)
پلاگین‌های کامپایلر	نسخه‌های سازگار با K2؛ پلاگین‌های Compose، serialization و Parcelize همراه Kotlin منتشر می‌شوند
تغییرات زبان	راهنمای سازگاری K2 را بخوانید؛ هشدارها را **قبل** از ارتقا رفع کنید
تحلیل استاتیک	نسخه‌هایی از detekt، ktlint و Lint که K2 را می‌فهمند
IDE	حالت K2 را در Android Studio یا IntelliJ روشن کنید تا IDE با کامپایلر یکی باشد

تلهٔ مصاحبه

«*K2 فقط یک کامپایلر سریع‌تر است.*» — یک frontend بازنویسی‌شده است. چند حالت مرزی در resolution و smart cast رفتار متفاوتی دارند، و همهٔ پلاگین‌های کامپایلر باید منتقل می‌شدند.

تلهٔ مصاحبه

«*API پلاگین K2 پایدار است — برای تولید کد، پلاگین کامپایلر بنویسید.*» — نسخهٔ پارسال همین را القا می‌کرد. API پلاگین کامپایلر **پایدار نیست** و بین نسخه‌های Kotlin می‌شکند. می‌خواهید کد تولید کنید؟ از KSP استفاده کنید (سؤال ۱۱).



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

KSP و متاپروگرمینگ، expect و actual، طراحی API عمومی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. انتخاب ابزار متاپروگرمینگ

ابزار	کِی اجرا می‌شود	چه می‌تواند	هزینه و ریسک
Reflection با `kotlin-reflect`	زمان اجرا	هر چیزی را بررسی کند	دسترسی اول کند، وابستگی بزرگ، قواعد keep در R8
kapt	زمان کامپایل، از طریق stubهای جاوا	اجرای annotation processorهای جاوا	تولید stub کند است؛ در حالت نگهداری
KSP	زمان کامپایل، با درک کاتلین	خواندن اعلان‌ها و تولید فایل‌های **جدید**	نمی‌تواند کد را تغییر دهد یا بدنهٔ تابع‌ها را ببیند
پلاگین کامپایلر	داخل کامپایلر	تبدیل کد، مثل Compose و serialization	API ناپایدار، بین نسخه‌های Kotlin می‌شکند
قاعدهٔ Lint یا detekt	تحلیل استاتیک	گزارش و اصلاح خودکار الگوها	تولید کد ندارد

تلهٔ مصاحبه

«*Dagger و kotlinx.serialization از reflection استفاده می‌کنند.*» — نسخهٔ پارسال همین را گفته بود. Dagger در زمان کامپایل کد تولید می‌کند؛ kotlinx.serialization از یک **پلاگین کامپایلر** استفاده می‌کند. برای همین سریع و سازگار با R8 هستند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. یک processor در KSP

```kotlin
class RouteProcessor(
    private val codeGenerator: CodeGenerator,
    private val logger: KSPLogger,
) : SymbolProcessor {
    override fun process(
        resolver: Resolver,
    ): List<KSAnnotated> {
        val (valid, deferred) = resolver
            .getSymbolsWithAnnotation("com.app.Route")
            .filterIsInstance<KSClassDeclaration>()
            .partition { it.validate() }
        valid.forEach { generateRoute(it) }
        return deferred // retried in the next round
    }
}

class RouteProcessorProvider : SymbolProcessorProvider {
    override fun create(env: SymbolProcessorEnvironment) =
        RouteProcessor(env.codeGenerator, env.logger)
}
```

- provider را در `META-INF/services` ثبت کنید؛ فایل‌ها را با KotlinPoet تولید کنید
- **incremental:** مقدار `Dependencies(aggregating = false, file)` را پاس بدهید تا فقط خروجی‌های مرتبط دوباره ساخته شوند
- خطاها را با `logger.error(msg, symbol)` گزارش کنید — به خط کد کاربر اشاره می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. expect و actual — نازک نگهش دارید

```
// commonMain
expect fun logDebug(tag: String, message: String)

interface SecureStore {
    suspend fun put(key: String, value: String)
    suspend fun get(key: String): String?
}

// androidMain
actual fun logDebug(tag: String, message: String) {
    android.util.Log.d(tag, message)
}
class KeystoreStore(ctx: Context) : SecureStore { /* … */ }

// iosMain
actual fun logDebug(tag: String, message: String) {
    println("[$tag] $message")
}
class KeychainStore : SecureStore { /* … */ }
```

- `expect fun` برای تابع‌های کوچک و برگ؛ **interface به‌علاوهٔ DI** برای سرویس‌ها — چون در تست‌های common می‌شود fakeشان کرد
- `actual typealias` یک expect را به یک تایپ موجود پلتفرم وصل می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. تله‌های Multiplatform

تلهٔ مصاحبه

رفتن سراغ `expect class` در قدم اول. **کلاس‌های** expect و actual هنوز Beta هستند و هشدار می‌دهند، مگر با `-Xexpect-actual-classes` آن را بپذیرید. یک interface که هر پلتفرم پیاده‌اش می‌کند ساده‌تر و تست‌پذیرتر است.

تلهٔ مصاحبه

نوشتن expect و actual برای چیزهایی که کتابخانه‌های common از قبل دارند: تاریخ (`kotlinx-datetime`)، ورودی و خروجی (`kotlinx-io`)، HTTP با Ktor، serialization. اول اکوسیستم را بگردید.

- **default hierarchy template** بدون سیم‌کشی دستی source setهای مشترک `iosMain` ، `appleMain` و `nativeMain` را می‌دهد
- بیشتر کد مشترک، کاتلین معمولی در `commonMain` است؛ expect و actual فقط لبهٔ نازک ماجراست — برای اشتراک ViewModelها روز ۳ را ببینید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. طراحی یک API عمومی در کاتلین

```
// build.gradle.kts
kotlin { explicitApi() }
```

```
@RequiresOptIn(message = "May change without notice")
public annotation class ExperimentalPaymentsApi

public class PaymentClient internal constructor(
    private val engine: Engine,
) {
    @ExperimentalPaymentsApi
    public suspend fun walletPay(r: WalletRequest): Receipt =
        engine.wallet(r)

    @Deprecated(
        "Use pay(PaymentRequest)",
        ReplaceWith("pay(PaymentRequest(amount))"),
        DeprecationLevel.ERROR,
    )
    public suspend fun pay(amount: Long): Receipt =
        pay(PaymentRequest(amount))
}
```

- **explicit API mode:** هر اعلان عمومی باید modifier دسترسی و تایپ صریح داشته باشد
- `@RequiresOptIn` برای بخش‌های ناپایدار؛ deprecate کردن مرحله‌به‌مرحله: اول `WARNING`، بعد `ERROR`، بعد `HIDDEN` — همراه با `ReplaceWith`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. تله‌های سازگاری باینری

```
public inline fun <reified T> Json.decodeOrNull(s: String): T? =
    decodeCatching { decodeFromString<T>(s) }

@PublishedApi
internal inline fun <T> decodeCatching(block: () -> T): T? =
    try { block() } catch (e: SerializationException) { null }
```

تلهٔ مصاحبه

یک **data class** در API عمومی. اضافه کردن یک property امضای constructor و `copy()` را عوض می‌کند — تغییری ناسازگار در سطح باینری برای هر کلاینتی که با نسخهٔ قبلی کامپایل شده. یک کلاس معمولی یا یک builder را ترجیح دهید.

تلهٔ مصاحبه

تابع inline عمومی که کد `internal` را صدا می‌زند. بدنهٔ inline در کد کلاینت کپی می‌شود، پس کامپایلر `@PublishedApi` را الزامی می‌کند — و آن تابع internal حالا برای همیشه بخشی از ABI شماست.

جواب سینیور در یک جمله

«بدان کامپایلر چه کاری برایت می‌کند و چه چیزی تا زمان اجرا می‌ماند: inline و reified کپی در محل فراخوانی‌اند، value classها در موقعیت generic باکس می‌شوند، contractها قول‌هایی‌اند که به آن‌ها اعتماد می‌شود — و KSP کد تولید می‌کند، هرگز آن را بازنویسی نمی‌کند.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۶ — خلاصهٔ یک‌صفحه‌ای

- `inline` بدنه و lambdaها را در محل فراخوانی کپی می‌کند؛ `noinline` object را **نگه می‌دارد**؛ `crossinline` بازگشت غیرمحلی را ممنوع می‌کند.
- generics در زمان اجرا erase می‌شوند؛ `reified` فقط در توابع inline.
- `when` ناقص روی تایپ sealed **خطاست** (از Kotlin 1.7)؛ به آن `else` اضافه نکنید.
- `out` یعنی تولیدکننده، `in` یعنی مصرف‌کننده؛ `MutableList<E>` ناورداست.
- value class در موقعیت generic، nullable یا `Any` باکس می‌شود.
- class delegation فقط فراخوانی را می‌فرستد؛ کارهای داخلی delegate به override شما نمی‌رسد.
- `@DslMarker` فقط receiverهای **ضمنی** بیرونی را می‌بندد.
- context parameterها: preview از Kotlin 2.2. به contractها اعتماد می‌شود، بررسی نمی‌شوند.
- K2 از Kotlin 2.0 پیش‌فرض است؛ به‌جای kapt از KSP استفاده کنید.
- KMP: لایهٔ نازک expect و actual. API عمومی: explicit API mode و بدون data class.

روز ۲۷ در راه است: الگوهای پیشرفتهٔ معماری. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700